ZIM School - Use Cases

01. Platform: accounts, sign-in, plans and announcements

Modules: Access, Billing, Announcements, Core
9 use cases (changes) · 5 views and reports
Written from the use case commands, handlers, policies and rules in `src/` on 2026-11-10.

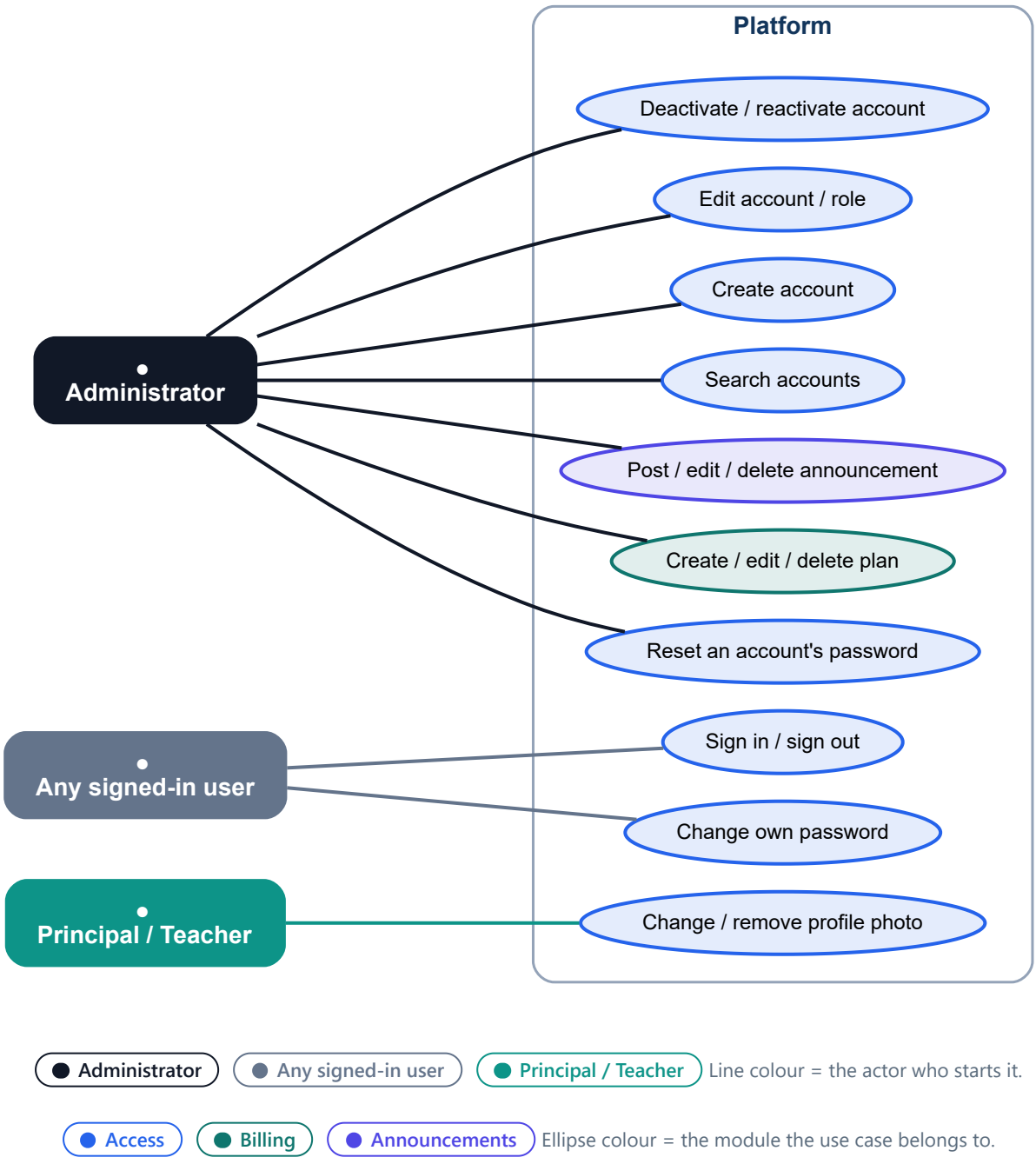
1. What this area does

The platform use cases manage **who can sign in** (accounts and passwords), **what a school may have** (plans) and **what the administrator tells everyone** (announcements). Every change in the system is written to the **activity log** (Core), which the admin dashboard shows as "Recent activity".

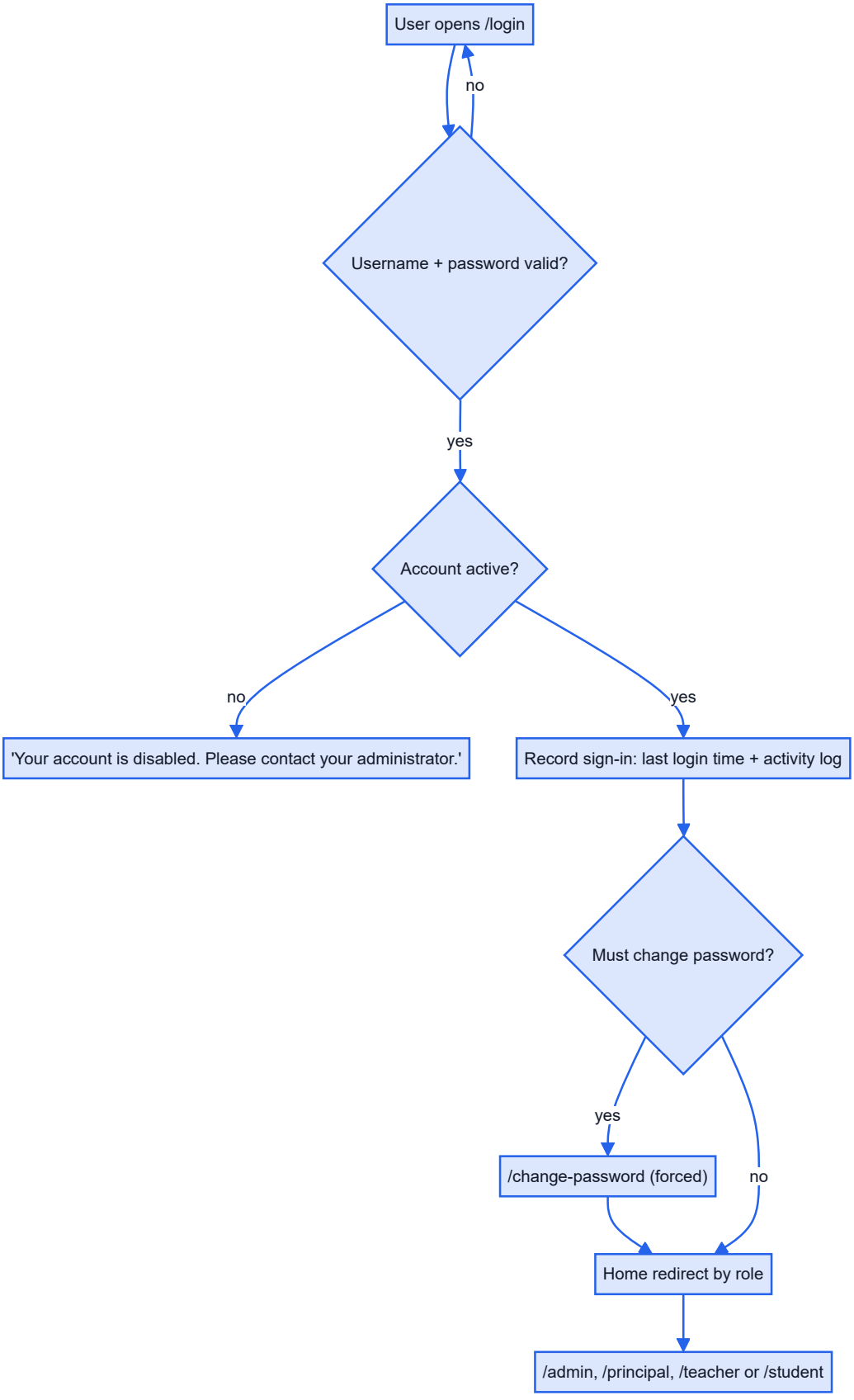
Accounts have exactly one role. Admins create principal and teacher accounts here (principals and admins can also create teacher accounts from a school's Teachers page); student accounts are created automatically when a learner is enrolled (username = LRN). Every new account must set its own password at first sign-in.

2. Use case map

A standard use case diagram: the actors on the left, each use case as an ellipse inside the system frame, and a line for each use case an actor starts. A dashed line has a note on when it applies.



3.1 Flow: Sign-in and first password



Access Drawn in the colour of the module this flow belongs to.

4. Use cases (changes)

Each card is one use case command sent through the use case bus. It runs in one database transaction: if any rule refuses, nothing is saved and the message in *Alternatives and refusals* is shown to the user.

PLT-01 Sign in

Primary actor	Any user
Goal	Get into the system with a username and password.
Where (route)	`app_login /login`
Preconditions	• The account exists and is active.
Main flow	1. The user enters username and password on the sign-in page. 2. Symfony Security checks the password hash and the account's active flag (UserChecker). 3. The system records the sign-in (RecordSignInCommand): sets the last login time and logs "Signed in". 4. If the account must change its password, the user is sent to Change password first. 5. Otherwise the home page (app_home) redirects to the dashboard of the user's role.
Alternatives and refusals	• Wrong username or password: the sign-in page shows an error. • Inactive account: "Your account is disabled. Please contact your administrator." • Forgot password: the page tells the user to ask the administrator to reset it (there is no e-mail reset).
Result (data changed)	User.last_login_at updated; ActivityLog row.
In the code	`RecordSignInCommand → RecordSignInHandler`

PLT-02 Change own password

Primary actor	Any signed-in user
Goal	Set a new personal password (forced at first sign-in or after a reset).
Where (route)	`app_change_password /change-password`
Preconditions	• The user is signed in.
Main flow	1. The user enters the current password and the new one twice. 2. The form checks the current password. 3. The system hashes and saves the new password, clears the must-change flag and logs "Changed password".
Alternatives and refusals	• The new password equals the current one: "Your new password must be different from your current one."
Result (data changed)	User.password replaced, must_change_password = false.
In the code	`ChangePasswordCommand`

PLT-03 Create an account

Primary actor	Administrator
Goal	Give a new person (admin, principal, teacher or student) a sign-in.
Where (route)	`admin_account_new /admin/accounts/new`
Preconditions	• Signed in as administrator.
Main flow	1. The admin fills in role, full name, username and a temporary password. 2. The system checks the username format and that it is free. 3. The account is saved with the temporary password hashed and must-change-password set. 4. The activity log records "Created account: <username>".
Alternatives and refusals	• Username taken: "This username is already taken." • Bad characters: "Use only letters, numbers, dots, dashes and underscores."
Result (data changed)	New User row.
In the code	`CreateAccountCommand. Console alternative: app:user:create`

PLT-04 Edit an account (including its role)

Primary actor	Administrator
Goal	Correct a person's name or username, change their role, or (de)activate them.
Where (route)	`admin_account_edit /admin/accounts/{id}/edit`
Preconditions	• Signed in as administrator; the account exists.
Main flow	1. The admin opens the account (GetAccountCommand shows the learner it belongs to, if any). 2. The admin changes the fields and saves. 3. Before a role change, other modules are asked through hooks whether it is allowed. 4. The account is saved and "Updated account" is logged.
Alternatives and refusals	• Username taken. • Role change refused, each reason shown under the role field, e.g. "This account belongs to learner <LRN>." (Learners) or "Remove <name> as principal of <school> (<year>) before changing the role." (Schools). • Reactivation refused by another module (e.g. no teacher seat left on the plan).
Result (data changed)	User updated.
In the code	`UpdateAccountCommand; hooks in Access/Domain/Service/Account`

PLT-05 Deactivate or reactivate an account

Primary actor	Administrator
Goal	Block or restore someone's access without deleting their history.
Where (route)	`admin_account_toggle_active /admin/accounts/{id}/toggle-active`
Preconditions	• Signed in as administrator.
Main flow	1. The admin presses Deactivate (or Activate) on the account list. 2. The system flips the active flag and logs it.
Alternatives and refusals	• Own account: "You cannot deactivate your own account." • Reactivation refused by a module hook (e.g. teacher seat limit).
Result (data changed)	User.active toggled. Deactivated accounts can no longer sign in.
In the code	`ToggleAccountActiveCommand`

PLT-06 Reset an account's password

Primary actor	Administrator
Goal	Give someone who forgot their password a new temporary one.
Where (route)	`admin_account_reset_password /admin/accounts/{id}/reset-password`
Preconditions	• Signed in as administrator; not the admin's own account.
Main flow	1. The admin presses Reset password. 2. The system generates a temporary password, saves its hash, sets must-change-password and logs it. 3. The temporary password is shown once on the page (pages are never prefetched, so it is not consumed early).
Alternatives and refusals	• Own account: "Use \"Change password\" to change your own password."
Result (data changed)	User.password replaced; must_change_password = true.
In the code	`ResetAccountPasswordCommand`

PLT-07 Change or remove the profile photo

Primary actor	Principal, Teacher
Goal	Show a photo on the account (also used on the staff ID).
Where (route)	`profile_photo /profile/photo; profile_photo_remove`
Preconditions	• Signed in as principal or teacher.
Main flow	1. The user uploads an image. 2. The image service re-encodes it to WebP and stores it under public/uploads (replacing the old file). 3. The path is saved and "Changed profile photo" is logged. Removing deletes the file and clears the path.
Alternatives and refusals	• Not an image / unreadable: "The file is not a readable image." • Too large: "The image is too large. Use one under 25 megapixels."
Result (data changed)	User.photo_path set or cleared.
In the code	`ChangeProfilePhotoCommand, RemoveProfilePhotoCommand`

PLT-08 Create, edit or delete a plan

Primary actor	Administrator
Goal	Define what schools can buy: teachers per seat, students per seat, whether a principal is included.
Where (route)	`admin_plans_new, admin_plans_edit, admin_plans_delete (/admin/plans/...)`
Preconditions	• Signed in as administrator. The Basic and Individual plans are seeded by migrations.
Main flow	1. The admin opens Plans, which lists each plan with how many schools use it. 2. Create/Edit: enter name, teachers per seat, students per seat and 'includes principal'; save. 3. Delete: allowed only when no school uses the plan. 4. Every change is logged.
Alternatives and refusals	• Name taken: "This plan already exists." • Delete while used: "<plan> is used by N school(s) and cannot be deleted."
Result (data changed)	Plan row created, updated or removed.
In the code	`CreatePlanCommand, UpdatePlanCommand, DeletePlanCommand`

PLT-09 Post, edit or delete an announcement

Primary actor	Administrator
Goal	Publish a notice for everyone, principals, teachers or students.
Where (route)	`admin_announcement_new / _edit / _delete (/admin/announcements/...)`
Preconditions	• Signed in as administrator.
Main flow	1. The admin writes a title, body and audience (Everyone, Principals, Teachers, Students). 2. The system saves it with the admin as author and the time, and logs it. 3. Edit changes the text or audience; Delete removes it.
Alternatives and refusals	• Announcement not found (deleted meanwhile): "The announcement was not found."
Result (data changed)	Announcement row created, updated or removed.
In the code	`PostAnnouncementCommand, UpdateAnnouncementCommand, DeleteAnnouncementCommand. Note: announcements are listed on the admin screens; no other module reads them yet.`

5. Views, downloads and reports (read only)

Each row is one query sent through the query bus. Queries change nothing.

Page / file	Who	Route	What it shows	Query
Account list (search, filter by role and active)	Administrator	`admin_accounts` `/admin/accounts`, `admin_accounts_by_role` `/admin/{slug}`	Paged accounts with each one's school this year.	`SearchAccountsCommand`
One account	Administrator	`admin_account_edit`	The account and, for a student, the learner's LRN.	`GetAccountCommand`
Plans	Administrator	`admin_plans` `/admin/plans`	Every plan with the number of schools on it.	`ListPlansCommand`
Announcements	Administrator	`admin_announcements` `/admin/announcements`	Every announcement, newest first.	`ListAnnouncementsCommand`
Recent activity	Administrator	`admin_dashboard /admin`	The newest lines of the activity log.	`GetRecentActivityCommand`